Tide retrospective: generic algebra for `gibbsExpectation` and `gibbsCov`

Daniel Murfet

7 May 2026

Abstract

A short infrastructure tide. The 1D and multivariate base Gibbs files in `laplace` carried the definitions of $\langle\cdot\rangle_t$ and $\mathrm{Cov}_t[\cdot,\cdot]$ but only the lone constant-collapse lemma $\langle c\rangle_t = c$. Every downstream consumer that wanted bilinearity, scalar pulling, or the constant case bashed through `unfold gibbsExpectation` and reproved the same algebraic identities by hand — most visibly in the affine-observable proofs in `OneD/Quartic.lean` (1D) and `TwoD/PureQuartic.lean` / `TwoD/SemiDegenerate.lean` (2D). This tide adds the missing algebra in parallel to both `Laplace/Gibbs.lean` and `Laplace/Multi/Basic.lean`: 11 lemmas each side, 245 lines total, single session, zero `sorry`, zero `native_decide`, zero `axiom`. The deliberation step converged on Candidate B (mirror across both base files) over A (1D-only) or C (B plus an affine corollary that belongs in I3). GPT-5.5 Pro flagged one strengthening worth taking — the constant-cov lemma is unconditional via a $Z = 0$ case-split rather than carrying a `hZ` hypothesis. No numerical sanity check was feasible (structural identities), and none was faked.

1 Setting

The cross-project survey of 7 May identified three classes of next moves on the laplace shoreline: strict improvements of recent tides, strategic refactors, and new-repo expansions. Item I2 sat in the strategic-refactor class: the GPT-5.5 Pro consult at Tide 12 (separable-potential abstraction) had recommended introducing `gibbsCov_symm`, `gibbsCov_add_left/right`, `gibbsCov_const_left/right`, and `gibbsCov_smul_left/right` in the base Gibbs files, observing that "once landed, the affine-covariance template becomes essentially trivial." The deferred I3 (an explicit affine-covariance theorem) and the strict-improvement candidates C1 (harmonic-side affine κ_3) and G2 (anharmonic-side affine κ_3) all wait on the same missing layer.

The seabed was `laplace` at commit `141997c` (post-G4)[1]. The two base files `Laplace/Gibbs.lean` (65 lines) and `Laplace/Multi/Basic.lean` (55 lines) defined `partitionFunction`, `gibbsExpectation`, `gibbsCov` and only `gibbsExpectation_const` (the 1D lone derived lemma; the multi side had no derived lemmas at all). A quick grep of consumer files confirmed the gap was load-bearing: the affine quartic covariance and its 2D pure-quartic analogue[2] reproved bilinearity inline at roughly 30 and 180 lines respectively, almost all bookkeeping.

The deliberation step[3] weighed three candidates: A (1D only, ∼120 lines), B (parallel both, ∼220), C (B plus the affine-bilinear corollary ∼280). Both Claude and GPT voted B. A is too

[1] Tide G4: `harmonic-gibbsobservable-monomials`, 397 lines; the most recent landed tide on `main` before this one. See `retrospectives/2026-05-07-tide-harmonic-gibbsobservable-monomials.tex`.

[2] Specifically, `gibbsCov_first_order_rate_sharp` in `Laplace/OneD/Quartic.lean` (lines 307–349) and the unnamed affine cov theorem in `Laplace/TwoD/PureQuartic.lean` (lines 297–475).

[3] Recorded in `projects/primer/tide-log/2026-05-07-tide-gibbscov-algebra.md` with the verbatim GPT-5.5 Pro response in `gpt55_tide_gibbscov_algebra_v1.md`.

small strategically (the heavy 2D consumers live on the multi side); C bundles a consumer-facing theorem that mixes in extra subtleties (Gibbs-weight integrability) and morally belongs in I3, not I2. B is one conceptual unit mirrored across the repo's two parallel foundations.

2 The thing formalised

The headline is not a single theorem but a small algebra package added in parallel to both base files. For the 1D file (the multi version is literal-substitution-equivalent), the eleven lemmas are:

```
namespace Laplace

lemma gibbsExpectation_smul (L : ℝ → ℝ) (t : ℝ) (c : ℝ) (φ : ℝ → ℝ) :
    gibbsExpectation L t (fun x => c * φ x) = c * gibbsExpectation L t φ

lemma gibbsExpectation_zero (L : ℝ → ℝ) (t : ℝ) :
    gibbsExpectation L t (fun _ => 0) = 0

lemma gibbsExpectation_add (L : ℝ → ℝ) (t : ℝ) (φ₁ φ₂ : ℝ → ℝ)
    (h₁ : Integrable (fun x => φ₁ x * Real.exp (-(t * L x))))
    (h₂ : Integrable (fun x => φ₂ x * Real.exp (-(t * L x)))) :
    gibbsExpectation L t (fun x => φ₁ x + φ₂ x)
      = gibbsExpectation L t φ₁ + gibbsExpectation L t φ₂

lemma gibbsCov_symm (L : ℝ → ℝ) (t : ℝ) (φ ψ : ℝ → ℝ) :
    gibbsCov L t φ ψ = gibbsCov L t ψ φ

lemma gibbsCov_smul_left  -- and _right by symm
lemma gibbsCov_const_left -- and _right by symm; unconditional
lemma gibbsCov_add_left   -- and _right by symm; needs four Integrable hyps
lemma gibbsCov_zero_left  -- and _right; cheap simp corollaries
```

The hypothesis economy is sharp: `symm` and the four `smul` lemmas need no hypotheses at all (only unconditional Bochner-integral linearity); `const` is unconditional thanks to the $Z = 0$ case-split (see §3); only `add_left/right` require the four `Integrable` hypotheses on the pre-divided weighted observables (each $\varphi_i \cdot e^{-tL}$ alone, plus each $\varphi_i \cdot \psi \cdot e^{-tL}$ for the cross-term inside the covariance). The same pattern is mirrored verbatim in `Laplace.Multi`, with $(\iota \to \mathbb{R})$ replacing $\mathbb{R}$.

3 Proof strategy

The covariance-level proofs all factor through the expectation-level primitives, which in turn route through three unconditional Mathlib lemmas: `MeasureTheory.integral_const_mul` (used by `gibbsExpectation_s`
`MeasureTheory.integral_add` (used by `gibbsExpectation_add`, the only place integrability is mathematically necessary), and field arithmetic (`mul_div_assoc`, `add_div`, `ring`). The covariance lemmas then follow by unfolding $\mathrm{Cov}_t[\varphi, \psi] = \langle \varphi\psi \rangle_t - \langle \varphi \rangle_t \langle \psi \rangle_t$ and rewriting with the expectation primitives plus a `ring`-style cleanup.

The minor wrinkle is the constant-cov lemma. The natural proof routes through $\langle c \rangle_t = c$, which holds only when $Z(t) \neq 0$. We could keep `hZ` as an explicit hypothesis, but as GPT-5.5 Pro pointed

out, when $Z(t) = 0$ Lean's `x / 0 = 0` convention collapses every Gibbs expectation to zero, so $\mathrm{Cov}_t[c, \psi] = 0 - 0 \cdot 0 = 0$ holds unconditionally. The proof case-splits:

```
by_cases hZ : partitionFunction L t = 0
· simp [gibbsCov, gibbsExpectation, partitionFunction] at hZ
  simp [hZ]
· simp only [gibbsCov]
  rw [..., gibbsExpectation_smul, gibbsExpectation_const L t c hZ]
  ring
```

The first branch flattens both the covariance and the partition function into the underlying integral, observes that the integral is zero, and lets `simp` propagate. The second branch is the usual substitution of $\langle c \rangle = c$. Both cleanly close.

The right-slot variants (`_right`) reduce to the left-slot ones by `gibbsCov_symm`. The right-slot integrability hypotheses for `gibbsCov_add_right` are stated in the natural orientation ($\varphi \cdot \psi_i \cdot e^{-tL}$) and reoriented by a one-line `simpa [mul_comm]` before delegating to `gibbsCov_add_left`.

4 Roadblocks and resolutions

Two minor frictions, both resolved on the first or second pass.

Argument-order on `gibbsCov_symm`. The first draft of `gibbsCov_add_right` used a bare `gibbsCov_symm` $\varphi\ \psi_2$ as a rewrite, mistakenly omitting the L and t arguments. Lean reported a type mismatch on the second positional argument. The fix was to spell the rewrites fully: `rw [gibbsCov_symm L t` $\psi_1\ \varphi$`, gibbsCov_symm L t` $\psi_2\ \varphi$`]`. Two minutes lost; promoted to the lessons section as a reminder that positional-argument economy is fragile when a downstream rewrite needs to target exactly two of three structurally-similar subterms.

Unconditional constant-cov. The first instinct was to keep `hZ` in `gibbsCov_const_left` and let a future cleanup handle the strengthening. GPT-5.5 Pro flagged the unconditional form as a free win: "if `partitionFunction L t = 0`, then every `gibbsExpectation ... / 0` collapses to `0`, so covariance with a constant is still `0`." The case-split proof above closed in one shot. Net cost: three lines beyond the natural-path proof; the value is one fewer hypothesis at every consumer site.

There were no thrashing episodes, no architectural pivots, and no counterexample-driven plan changes. The lemmas are mechanical from the definitions; the deliberation step did the substantive work.

5 What was Mathlib, what was new

Mathlib provided. `MeasureTheory.integral_const_mul` (real-valued, `RCLike` instance, unconditional) and `MeasureTheory.integral_add` (the conditional additivity that forces the four `Integrable` hypotheses on the cov-add lemmas). Plus the usual algebraic background: `mul_comm`, `mul_div_assoc`, `add_div`, `ring`.

What was new. The algebra layer itself: 11 lemmas per side, 245 lines total. No new Mathlib lemmas were upstreamed because nothing in this Tide is general enough to belong in Mathlib — `gibbsCov` is repo-specific. The shape of the layer might nonetheless be worth lifting, eventually:

a generic `Lebesgue.expectation` / `Lebesgue.cov` pattern with parallel algebra lemmas for any σ-finite measure could replace both the 1D and multi versions in the laplace repo, plus possibly threepoint's `Threepoint.gibbsCov`. Not in scope here.

6 Lessons

- *For the laplace `CLAUDE.md`:* the `x / 0 = 0` case-split idiom for unconditional algebra lemmas on quotient definitions like `gibbsExpectation` = `integral/Z`. The recipe `by_cases hZ : Z = 0` followed by `simp [..., partitionFunction] at hZ ⊢; simp [hZ]` on the zero branch and the natural-path proof on the nonzero branch is short and idiomatic. Worth landing as a tactic note alongside the existing `Pi.add` and `rpow_natCast` entries.

- *For the lean-formalisation skill:* the parallel-mirror pattern. The two base files differ only by the underlying domain ($\mathbb{R}$ versus $\iota \to \mathbb{R}$); the proofs are literal-substitution-equivalent because the underlying Mathlib integration lemmas are parametric in the measurable space. Writing the 1D version first, getting it to compile, and then doing a mechanical copy-paste-rename for the multi side cost roughly half again the time of doing only one side. This pattern recurs across the laplace repo (`Quartic.lean` versus `TwoD/PureQuartic.lean`, `Harmonic.lean` versus `TwoD/AddSeparable.lean`); the duplication is real but predictable and worth the cost.

- *For the tide skill:* the structural-identity case for "Numerical sanity check." This Tide produced no closed-form expectation (the lemmas *are* structural identities, true by `rfl`/`ring` from the definitions). The skill's fallback "not feasible: *reason*" was exactly the right move; the alternative — fabricating a numerical check — would have wasted time and produced a false signal. The tide-log entry records the reason explicitly, so the next survey can see why the section is short.

7 Follow-ups

- *I3 — affine-bilinear covariance.* Now a 5–10 line derivation:

$$\mathrm{Cov}_t[a\varphi + c,\, b\psi + d] = ab \cdot \mathrm{Cov}_t[\varphi, \psi]$$

  follows by two applications each of `gibbsCov_add_left`, `gibbsCov_const_left`, `gibbsCov_smul_left`, and their right-slot symmetries. Worth a small dedicated tide that pins the integrability hypotheses cleanly and exports it as a derived theorem in `Laplace/Gibbs.lean` (and `Multi/Basic.lean`).

- *C1 — harmonic-side affine κ_3.* Generalise from $\kappa_3(x, x, x)$ to $\kappa_3(b \cdot x, a \cdot x, c \cdot x)$ via multilinearity of κ_3. With the new `gibbsCov_add/smul` algebra and the existing harmonic κ_3 identity, $\sim$50 lines on top of `Threepoint/Harmonic.lean`.

- *G2 — anharmonic-side affine κ_3.* Same shape as C1 but for the anharmonic Gibbs measure. $\sim$40 lines on top of `Laplace/OneD/AnharmonicKappa3.lean`.

- *Cleanup pass on existing affine-cov proofs.* The most satisfying use of this Tide will be the line-count reductions in the four files[4] that currently unfold `gibbsExpectation` to reach the same algebraic identities now captured by the new lemmas. A focused refactor tide (or a sequence of small ones) would tighten $\sim$300 lines of bookkeeping into $\sim$50. Not strictly necessary — the existing proofs work — but cost-positive once the algebra is in.

[4] `Laplace/OneD/Quartic.lean`, `Laplace/TwoD/PureQuartic.lean`, `Laplace/TwoD/SemiDegenerate.lean`, `Laplace/TwoD/AddSeparable.lean`.

- *Optional: bundle the hypotheses.* If the `Integrable`-quadruples on the cov-add lemmas turn out to be the dominant signature noise at consumer sites (likely after C1, G2, and I3 land), a plain-`Prop` alias `GibbsAddIntegrable L t` φ ψ bundling the four hypotheses would compress signatures without committing to typeclass plumbing. Defer until the second downstream use shows it's worth it.

Acknowledgements

The deliberation step's GPT-5.5 Pro consult (preserved at `projects/primer/tide-log/gpt55_tide_gibbscov_alg`) contributed the unconditional-const strengthening, the direct-hypotheses-over-typeclass call, and the explicit reason to prefer Candidate B over A or C. The Tide 12 deliberation (separable-potential abstraction, 6 May) is where the I2 candidate originated; this Tide closes a follow-up that has been on the board for less than a day.